

Ledge Identity — Product Requirements Document

Version: 1.0 **Date:** 2026-10-02 **Author:** PRD Generator **Status:** Draft

Assumptions used in this document (override any of them before build): - Working product name is "Ledge Identity" (taken from the source draft). - Stack: Next.js + Node.js/TypeScript API + PostgreSQL, S3-compatible storage, queue workers, Clerk (auth), Stripe (payments), hosted on Vercel (web) and Railway/AWS (workers). - Image/video generation is routed through Higgsfield and Krea, behind a provider-agnostic abstraction. - Release target is a closed alpha of 50–100 invited users, with a 60-day build. - MVP is self-identity only, web only (mobile web capture), launching to adult users in one jurisdiction (United States, excluding states with unresolved biometric-law review — see Open Questions). - REST API style. All times UTC.

1. Executive Summary

Ledge Identity lets creators, musicians, influencers and small-business owners turn their own photos into a persistent, user-controlled **Identity Profile** and then reuse it to generate recognizable images and short animated clips across styles, scenes and formats. It solves the core failure of general image generators: the person does not look like themselves from one output to the next, and the workflow is split across many disconnected tools. The MVP delivers consent-first identity creation, identity-consistent image generation, a credit-based economy, and basic safety and deletion controls. The expected outcome is that a new user gets a recognizable identity-based image within 15 minutes of a successful upload and keeps creating weekly.

2. Problem Statement

Current state: Creators who want professional visuals of themselves rely on photo shoots, crews and editors, or on generic AI image tools that restart from scratch with every prompt. Headshots, avatars, images, video animation, editing and publishing each live in a separate product, and none of them keeps a durable, user-owned identity layer.

Pain points:

1. Generated people inconsistently resemble the user (face structure, skin tone, hair, proportions, age).
2. Producing frequent content is slow and expensive without shoots, wardrobe, locations and post-production.
3. Prompt engineering is required to get acceptable results, and results cannot be reproduced.
4. Image, animation and publishing workflows are fragmented across tools, with no shared identity or lineage.
5. Users have no clear control over how their likeness data is stored, used, shared or deleted.

Impact: Creators publish less often and spend 5+ hours per asset on traditional production or re-prompting. Businesses lose speed in testing creative. For the platform, an untrusted likeness product carries critical legal and reputational exposure (biometric privacy, impersonation, non-consensual content), so trust is a product requirement, not a feature.

3. Goals & Success Metrics

Goal	Metric	Target (closed alpha)	Measurement Method
Fast time to first value	Median time from upload complete to first approved identity-based image	≤ 15 min	Event timestamps: <code>identity_upload_completed</code> → <code>generation_completed</code>
Users get an approved identity	Identity approval rate (approved ÷ identity creations started)	≥ 70%	<code>identity_approved</code> ÷ <code>identity_creation_started</code>
Outputs are recognizable	Users rating identity fidelity 4 or 5 of 5	≥ 65%	In-app rating after first generation
Users complete the core loop	Activated Creator rate (identity approved AND first generation completed AND first asset saved/shared)	≥ 40% of signups	Funnel in analytics
Repeated use	Weekly Identity-Powered Creations per active creator (north star)	≥ 6	Count of <code>generation_completed</code> with verified identity
Retention	D7 creator retention	≥ 25%	Cohort analysis
Reliable generation	Generation success rate	≥ 95%	Job status counts
Sustainable economics	Gross margin per generated image after provider cost	≥ 60%	Provider cost logged per job vs. credit price
Trust	Identity deletions completed within SLA	100% within 72 h of request	Admin audit log

Non-goals (MVP): feature-film-grade production; autonomous publishing to social networks; marketplace for selling likeness; celebrity, public-figure or third-party identities; voice cloning, real-time face swap or live impersonation; enterprise team admin/SSO; native mobile apps; a general video editor.

4. User Personas

Independent Visual Creator (primary)

- **Role:** Musician, influencer, podcaster, educator, streamer or artist
- **Goals:** Publish fresh, consistent visuals of themselves frequently without a shoot
- **Pain points:** Inconsistent AI likeness, too many tools, high production cost
- **Technical proficiency:** Medium
- **Usage context:** Mobile web for photo capture, desktop for creating and exporting; several sessions per week

Brand Operator (secondary)

- **Role:** Founder, coach, realtor, consultant or e-commerce seller
- **Goals:** Produce ad-ready imagery with a recognizable personal-brand spokesperson, test creative quickly
- **Pain points:** Agency cost, slow iteration, inconsistent brand look
- **Technical proficiency:** Low to Medium
- **Usage context:** Campaign bursts, usually desktop, exports to ad platforms manually

Creative Collaborator (tertiary, post-MVP)

- **Role:** Social-media manager, editor or freelancer working for an identity owner
 - **Goals:** Produce assets from an approved identity without unrestricted account access
 - **Pain points:** Sharing logins, no permission model
 - **Technical proficiency:** Medium to High
 - **Usage context:** Scoped project access granted by the owner
-

5. Functional Requirements

Priority split: 7 of 20 features are P0 (35%), 8 are P1 (40%), 5 are P2 (25%).

FR-001: Account & Age Gate

Description: Users create an account and confirm they meet the minimum age before any identity creation.

User story: As a creator, I want to sign up quickly and securely so that I can start building my identity.

Acceptance criteria:

- [] Sign-up supports email/password and Google OAuth via Clerk
- [] User must confirm they are 18 or older before the identity flow becomes available; the confirmation is stored with timestamp and policy version
- [] Users can sign out, reset credentials and request account deletion
- [] A user cannot read another user's identity, assets, wallet or source photos (verified by automated authorization tests returning 403/404)
- [] Each account gets one private default workspace on first sign-in
- [] Login and password-reset endpoints are rate limited to 10 requests per minute per IP

Priority: P0

Dependencies: None

FR-002: Identity Consent Capture

Description: Before uploading photos, users complete a standalone likeness-consent step, separate from the general terms of service.

User story: As a creator, I want to understand and control how my likeness is used so that I can trust the platform.

Acceptance criteria:

- [] Consent screen states purpose, processing, storage, generation, sharing, retention and deletion in plain language, under 400 words
- [] User must check an explicit attestation: "I am the person in these photos, or I have legal authority to use them"
- [] Consent record stores user ID, identity ID, policy version, attestation text, IP address hash and timestamp, and is immutable
- [] Default setting: source photos and identity artifacts are NOT used to train general models; any such use requires a separate opt-in toggle, off by default
- [] User can withdraw consent from settings; withdrawal disables generation with that identity immediately
- [] Users cannot proceed to upload without a stored consent record

Priority: P0

Dependencies: FR-001

FR-003: Photo Upload & Validation

Description: Users upload 10–25 photos and receive per-image quality validation with corrective guidance.

User story: As a creator, I want clear feedback on my photos so that I can fix problems before spending time on identity creation.

Acceptance criteria:

- [] Accepted formats: JPG, JPEG, PNG, HEIC, WEBP; max 15 MB per file; minimum 1024 px on the shorter side
- [] Minimum 10 and maximum 25 images per identity; recommended 15+
- [] Per-image validation completes in under 5 seconds after upload and flags: no face, multiple faces, blur, heavy filter, sunglasses/occlusion, near-duplicate, low resolution
- [] Each rejected image shows the specific reason and one suggested fix
- [] Upload progress is shown per file; interrupted uploads can resume without restarting completed files
- [] Mobile web users can capture directly from the device camera with an on-screen guide (lighting, angle, expression variety)
- [] Originals are stored encrypted, owner-only, and never publicly accessible

Priority: P0

Dependencies: FR-002

FR-004: Identity Profile Creation & Approval

Description: The system builds a persistent, versioned Identity Profile from validated photos and lets the user approve it using a preview.

User story: As a creator, I want a reusable identity so that every generation looks like me.

Acceptance criteria:

- [] Creating an identity enqueues an async job; status moves through draft → processing → review_required | approved | failed
- [] Identity creation completes in under 15 minutes under normal queue conditions
- [] On completion the user sees an "identity check" preview of 4 images and can Approve, Retry with different photos, or Request review
- [] Each approved identity has an immutable ID and an incrementing identity_version; improved photo sets create a new version, old versions stay available for lineage
- [] Identities that fail the automated quality threshold route to a manual review queue visible in the admin console
- [] Platform-caused failure grants a free retry (no credit charge)
- [] MVP limit: one active verified identity per account

Priority: P0

Dependencies: FR-003, FR-006

FR-005: Identity-Consistent Image Generation

Description: Users generate images featuring their approved identity from a template or prompt, with the credit cost shown before confirmation.

User story: As a creator, I want to generate images that look like me in any setting so that I can publish without a shoot.

Acceptance criteria:

- [] User selects an approved identity, a world/template or free prompt, aspect ratio, and 1, 2 or 4 variations
- [] Exact credit cost is displayed before the user confirms; confirming reserves credits atomically
- [] Standard image job completes in under 90 seconds at p90 under normal load
- [] Job statuses shown to the user: queued, preparing_identity, generating, enhancing, completed, failed
- [] Completed outputs are saved to the library automatically with lineage (identity version, template, prompt, provider, model version, settings, cost)
- [] Prompts and outputs pass moderation before release; blocked requests show a policy-aligned explanation and no credit is consumed
- [] Failed jobs return reserved credits automatically within 60 seconds
- [] Duplicate submissions with the same idempotency key create one job only

Priority: P0

Dependencies: FR-004, FR-006, FR-007

FR-006: Credit Wallet & Ledger

Description: A wallet tracks credits by type with an immutable ledger of every grant, reservation, deduction, refund and expiration.

User story: As a creator, I want to know exactly what each action costs and what I have left so that I'm never surprised.

Acceptance criteria:

- [] New accounts receive a configurable starter grant (default 100 credits) with a stated expiry
- [] Wallet displays available, reserved, purchased, bonus and subscription balances separately, plus next expiration date
- [] Reservation, settlement and refund happen in a single database transaction linked to the job; a job can never be charged twice (idempotency key enforced)
- [] Balance can never go negative
- [] Every ledger entry is append-only with type, amount, source, reference ID, expiration and timestamp
- [] Credit costs are configuration values, not hard-coded, and change without a deploy

Priority: P0

Dependencies: FR-001

FR-007: Moderation, Takedown & Identity Deletion

Description: Automated and manual safety controls, plus self-service deletion of identities and accounts.

User story: As a creator and as the platform operator, I want likeness misuse prevented and my data deletable so that the product is safe to use.

Acceptance criteria:

- [] Prompts are moderated before generation and outputs after generation; prohibited categories (sexual content involving real likeness, minors, deceptive impersonation, defamation) are blocked
- [] Any asset has a "Report" action; reports create a Safety Case visible in the admin console
- [] Admins can suspend an account, disable an identity and remove an asset, each producing an audit event
- [] User-initiated identity deletion disables generation instantly and permanently removes source photos, identity artifacts and derived outputs within 72 hours, including provider-side deletion requests; deletion completion is logged
- [] Account deletion cascades to all identities, assets and personal data, except ledger and payment records retained as legally required
- [] Only self-identity is permitted; creation attempts flagged as celebrities or minors are blocked or sent to manual review

Priority: P0

Dependencies: FR-004

FR-008: World Explorer

Description: A browsable catalog of styles, scenes and formats so users don't need prompt expertise.

User story: As a creator, I want ready-made worlds so that I can get a great result without writing prompts.

Acceptance criteria:

- [] Catalog has at least 30 worlds at alpha across categories: headshots, editorial, film stills, sci-fi/fantasy, cartoon/anime, music-video/album-cover, UGC/ads, lifestyle
- [] Each world shows thumbnail, title, supported aspect ratios, credit estimate and a starter prompt
- [] Users can filter by category and search by keyword with results in under 2 seconds
- [] Selecting a world pre-fills the generation form; every field remains editable
- [] Templates are versioned; a published generation records the template version used

Priority: P1

Dependencies: FR-005

FR-009: Generation History & Library

Description: A searchable library of every job and asset with settings and cost.

User story: As a creator, I want to find and reuse past work so that I can build on it.

Acceptance criteria:

- [] Library lists all assets with filters: date, project, type (image/video), world, status, favorite
- [] Opening an asset shows prompt, settings, identity version, provider, credit cost and job status
- [] Users can organize assets into named projects, duplicate a project and bulk-delete assets
- [] Filter/search results return in under 2 seconds for libraries up to 5,000 assets
- [] Deleted assets are removed from storage and CDN within 24 hours

Priority: P1

Dependencies: FR-005

FR-010: Remix, Variations & Upscale

Description: Users iterate on completed outputs by changing pose, style or background, or by upscaling.

User story: As a creator, I want to refine a result I like so that I don't start from zero.

Acceptance criteria:

- [] "Remix" opens the generation form pre-filled from a previous output, preserving reference lineage
- [] Any completed output can be used as an image reference in a later generation
- [] Upscale produces at least 2× resolution and costs a displayed credit amount
- [] Remix and upscale jobs follow the same status, refund and moderation rules as FR-005

Priority: P1

Dependencies: FR-005

FR-011: Subscriptions & Credit Packs

Description: Stripe-powered recurring plans and one-time credit packs.

User story: As a creator, I want to buy more credits or a plan so that I can keep creating.

Acceptance criteria:

- [] Plans: Free, Creator, Pro, defined in configuration (Studio is post-MVP)
- [] Subscription renewals grant credits exactly once per billing period (webhook idempotency by Stripe event ID)
- [] One-time packs credit the wallet within 30 seconds of a confirmed payment
- [] Users can view invoices, change plan, cancel and update payment method via Stripe customer portal
- [] Replayed or out-of-order Stripe webhooks never double-grant credits

Priority: P1

Dependencies: FR-006

FR-012: Image-to-Video Animation

Description: Users animate an approved identity-based image into a short video clip.

User story: As a creator, I want to turn a still into motion so that I can make shareable short-form content.

Acceptance criteria:

- [] Duration options: 5, 8, 10 seconds (subject to provider support); aspect ratios 9:16, 1:1, 16:9
- [] Motion templates: subtle portrait, camera push-in, turn, walk, performance energy, ambient scene
- [] Credit cost and expected completion range are shown before confirmation; premium confirmations require an extra explicit click
- [] Video job completes in under 10 minutes at p90
- [] Failed provider job after retries refunds credits and shows a clear failed state
- [] Output retains lineage to source image, identity version and workflow

Priority: P1

Dependencies: FR-005, FR-007

FR-013: Export & Share Pages

Description: Download assets and create revocable share pages.

User story: As a creator, I want to download and share my work so that it reaches my audience.

Acceptance criteria:

- [] Images download as PNG/JPG, videos as MP4
- [] Share pages support three visibility modes: private link, unlisted, public; default is private link
- [] Share links can be revoked at any time and stop resolving within 60 seconds
- [] Free-plan exports include a visible or metadata disclosure per FR-015

Priority: P1

Dependencies: FR-009, FR-015

FR-014: Referral Rewards

Description: Unique referral links that grant credits after a referred user activates.

User story: As a creator, I want to earn credits for inviting friends so that I can make more content.

Acceptance criteria:

- [] Each user has one unique referral link
- [] Reward is issued only after the referred account reaches Activated Creator state
- [] Self-referral and duplicate-account patterns (same device fingerprint, payment method, or IP cluster) are blocked or held for manual review
- [] Referral credits have a stated amount and expiry, and appear as bonus in the ledger
- [] Referral does not require the user to make any identity or content public

Priority: P1

Dependencies: FR-006

FR-015: Provenance & Disclosure Metadata

Description: Generated media carries machine-readable and, where required, visible AI-generated indicators.

User story: As a creator and as a platform, I want outputs clearly marked as AI-generated so that they cannot be mistaken for authentic recordings.

Acceptance criteria:

- [] Every exported asset embeds provenance metadata (C2PA manifest or equivalent) with generator name, date and identity-owner ID hash
- [] Free-plan exports carry a visible watermark; paid plans can toggle visible disclosure but metadata cannot be removed

- ☐ Share pages display an "AI-generated" label

Priority: P1

Dependencies: FR-005

FR-016: Brand Kits

Description: Save colors, mood references and recurring worlds as reusable presets.

User story: As a brand operator, I want saved style presets so that campaigns stay consistent.

Acceptance criteria:

- ☐ A brand kit stores up to 8 colors, 10 reference images, a mood description and up to 5 favorite worlds
- ☐ Selecting a kit in the generation form applies its settings as defaults

Priority: P2

Dependencies: FR-008

FR-017: Collaborator Access

Description: Owners invite a collaborator with scoped access to a project.

User story: As an identity owner, I want a collaborator to create assets without sharing my login so that I keep control of my likeness.

Acceptance criteria:

- ☐ Owner can invite by email, grant access to one project, and revoke at any time
- ☐ Collaborators can generate with the owner's identity inside that project only, spending owner credits at owner-set limits
- ☐ Collaborators cannot view source photos, export the identity, or delete it

Priority: P2

Dependencies: FR-009

FR-018: Social Publishing & Scheduling

Description: Direct publish or schedule to social platforms.

User story: As a creator, I want to publish directly so that I save export steps.

Acceptance criteria:

- ☐ OAuth connection to at least one platform with revoke option
- ☐ Scheduled posts show status and failure reasons

Priority: P2

Dependencies: FR-013

FR-019: Multi-Scene Video Editor

Description: Timeline editing, transitions and audio sync across clips.

User story: As a creator, I want to assemble clips into a sequence so that I can make a longer piece.

Acceptance criteria:

- [] Users can arrange at least 5 clips on a timeline and export one MP4
- [] Basic transitions and a single audio track are supported

Priority: P2

Dependencies: FR-012

FR-020: Multiple Identities per Account

Description: Additional identities (e.g., professional, character versions) under reviewed conditions.

User story: As a creator, I want separate identities for different looks so that I can keep them organized.

Acceptance criteria:

- [] Up to 3 identities per account, each with its own consent record
- [] Each additional identity passes the same validation and review as the first

Priority: P2

Dependencies: FR-004

6. Non-Functional Requirements

Performance

- Page load time: under 3 seconds on typical broadband
- API response time: p95 under 400 ms for non-generation endpoints
- Photo validation feedback: under 5 seconds per image
- Standard image generation: under 90 seconds p90; video: under 10 minutes p90
- Concurrent users supported: 200 at alpha, 2,000 at public beta

Security

- Authentication method: Clerk-issued JWT, verified server-side on every request
- Data encryption: TLS 1.2+ in transit; AES-256 at rest for source photos, identity artifacts and tokens
- Authorization enforced server-side on every identity, project, asset, export and wallet action
- Rate limits on uploads, generation, login, password reset and referral actions
- Provider API keys, payment credentials and internal identity references never reach the client
- Compliance requirements: GDPR/UK GDPR principles, CCPA, and biometric privacy laws (e.g., Illinois BIPA, Texas CUBI, Washington) treated as applicable until legal review clears each jurisdiction

Scalability

- Expected initial load: 100 users, ~2,000 generation jobs per week
- Growth target: 5,000 users and 100,000 jobs per month within 6 months
- Scaling approach: stateless API (horizontal), independent worker pools per job type, queue-based back-pressure

Availability

- Uptime target: 99.5% monthly for MVP
- Backup frequency: database every 6 hours with point-in-time recovery; storage versioning on non-identity assets
- Disaster recovery: RTO 4 hours, RPO 6 hours; backups must honor deletion (deleted identity data purged from backups within 30 days, documented in the retention policy)

Accessibility

- WCAG 2.1 AA for onboarding, upload, creation, export, account and deletion flows
 - Keyboard navigation, labeled form controls, screen-reader-announced job status, no color-only status
-

7. Technical Architecture

System Overview

A Next.js web client talks to a TypeScript/Node API behind Clerk authentication. The API validates and authorizes every request and writes jobs to a Redis-backed queue. Separate worker services process identity creation, image generation, animation, moderation and refunds by calling provider adapters (Higgsfield, Krea). Media is stored in S3-compatible storage with signed URLs and a CDN. PostgreSQL holds all relational data including the credit ledger.

Technology Stack

- **Frontend:** Next.js 15 (App Router), React, TypeScript, Tailwind CSS
- **Backend:** Node.js 22, TypeScript, Fastify (or Express), Zod for validation
- **Database:** PostgreSQL 16 (Prisma ORM)
- **Queue:** Redis + BullMQ
- **Storage:** S3-compatible bucket with CloudFront/CDN and signed URLs
- **Auth:** Clerk
- **Payments:** Stripe (Checkout, Billing, Customer Portal, webhooks)
- **Hosting:** Vercel (web), Railway or AWS ECS (API and workers)
- **Key libraries:** TanStack Query, Sharp (image checks), Playwright (E2E), Vitest (unit)

Architecture Diagram (Description)

"A user's browser loads the Next.js app from Vercel. Browser calls go to the API service with a Clerk JWT. Photo uploads go straight from the browser to S3 using pre-signed URLs, after the API records the intent. The API creates a job row in PostgreSQL, reserves credits in the same transaction, then enqueues the job in BullMQ. A worker picks up the job, runs input moderation, calls the selected provider adapter, runs output moderation, writes the asset to S3, updates the job, and settles or refunds credits. The client polls job status (or receives it over server-sent events). Stripe webhooks hit a dedicated endpoint that verifies signatures and writes ledger entries idempotently. An admin console, protected by role-based access, reads Safety Cases, identity reviews and audit events."

8. API Specifications

All endpoints are prefixed /api/v1. Authenticated endpoints require Authorization: Bearer <Clerk JWT>.

POST /api/v1/identities

Purpose: Start creating an identity after consent and uploads.

Authentication: Required

Request:

```
{
  "name": "string – private label, 1-60 chars",
  "consent_id": "uuid – consent record from POST /consents",
  "source_asset_ids": ["uuid – 10 to 25 validated source assets"]
}
```

Response (202):

```
{
  "identity_id": "uuid",
  "status": "processing",
  "identity_version": 1,
  "estimated_ready_at": "ISO-8601 timestamp"
}
```

Error responses:

- 400: Fewer than 10 or more than 25 valid source assets
- 401: Missing or invalid token
- 403: Consent record belongs to another user, or age gate not passed
- 409: User already has an active identity (MVP limit)

POST /api/v1/consents

Purpose: Record likeness consent before upload.

Authentication: Required

Request:

```
{
  "policy_version": "string – e.g. 2026-10-01",
  "attestation_accepted": "boolean – must be true",
  "allow_model_training": "boolean – default false"
}
```

Response (201):

```
{
  "consent_id": "uuid",
  "recorded_at": "ISO-8601 timestamp"
}
```

Error responses:

- 400: attestation_accepted is not true
- 401: Not authenticated

POST /api/v1/uploads

Purpose: Get a pre-signed URL for a source photo and register it for validation.

Authentication: Required

Request:

```
{
  "filename": "string",
  "content_type": "string – image/jpeg | image/png | image/heic | image/webp",
  "size_bytes": "integer – max 15728640",
  "consent_id": "uuid"
}
```

Response (201):

```
{
  "asset_id": "uuid",
  "upload_url": "string – pre-signed, expires in 10 minutes",
  "validation_status": "pending"
}
```

Error responses:

- 400: Unsupported type or file too large
- 403: No valid consent record
- 429: Upload rate limit exceeded

POST /api/v1/generations

Purpose: Create an image generation job and reserve credits.

Authentication: Required

Request:

```
{
  "identity_id": "uuid",
  "world_id": "uuid | null",
  "prompt": "string – max 1000 chars",
  "aspect_ratio": "string – 1:1 | 4:5 | 9:16 | 16:9",
  "variations": "integer – 1, 2 or 4",
  "settings": {
    "identity_strength": "number – 0.0 to 1.0",
    "style_intensity": "number – 0.0 to 1.0"
  },
}
```

```
"idempotency_key": "string – client-generated UUID"
}
```

Response (202):

```
{
  "job_id": "uuid",
  "status": "queued",
  "credits_reserved": 16,
  "balance_available": 84
}
```

Error responses:

- 400: Invalid settings
- 402: Insufficient available credits
- 403: Identity not approved, suspended, or owned by another user
- 422: Prompt blocked by moderation (response includes `reason_code` and `suggestion`)
- 429: Generation rate limit exceeded

GET /api/v1/generations/{job_id}

Purpose: Poll job status and fetch outputs.

Authentication: Required

Response (200):

```
{
  "job_id": "uuid",
  "status": "queued | preparing_identity | generating | enhancing | completed | failed",
  "assets": [
    { "asset_id": "uuid", "url": "string – signed, expires in 1 hour", "type": "image" }
  ],
  "credits_charged": "integer",
  "error": { "code": "string", "message": "string" }
}
```

Error responses:

- 401: Not authenticated
- 404: Job not found or not owned by caller

POST /api/v1/animations

Purpose: Create an image-to-video job.

Authentication: Required

Request:

```
{
  "source_asset_id": "uuid",
  "motion_template": "string – e.g. camera_push_in",
  "duration_seconds": "integer – 5, 8 or 10",
  "aspect_ratio": "string",
  "confirm_premium": "boolean",
  "idempotency_key": "string"
}
```

Response (202):

```
{
  "job_id": "uuid",
  "status": "queued",
  "credits_reserved": 30,
  "estimated_completion_seconds": 420
}
```

Error responses:

- 402: Insufficient credits
- 403: Source asset not owned or not eligible
- 422: Blocked by moderation

DELETE /api/v1/identities/{identity_id}

Purpose: Delete an identity and start permanent data removal.

Authentication: Required

Response (202):

```
{
  "identity_id": "uuid",
  "status": "deletion_in_progress",
  "generation_disabled": true,
  "deletion_deadline": "ISO-8601 timestamp (within 72 hours)"
}
```

Error responses:

- 403: Not the owner
- 404: Identity not found

GET /api/v1/wallet

Purpose: Return balances and recent ledger entries.

Authentication: Required

Response (200):

```
{
  "available": "integer",
  "reserved": "integer",
  "by_type": { "purchased": 0, "bonus": 0, "subscription": 0 },
  "next_expiration": { "amount": 0, "expires_at": "ISO-8601 timestamp" },
  "recent_entries": [
    { "id": "uuid", "type": "reservation | settlement | refund | grant | expiration", "amount": 0,
  "created_at": "ISO-8601 timestamp" }
  ]
}
```

Error responses:

- 401: Not authenticated

POST /api/v1/webhooks/stripe

Purpose: Receive Stripe events and update subscriptions and credits.

Authentication: None (verified by Stripe signature header)

Request: Raw Stripe event body

Response (200):

```
{ "received": true }
```

Error responses:

- 400: Invalid signature
- 200 with "duplicate": true if the event ID was already processed

9. UI/UX Requirements

Landing / Sign-in

Purpose: State the value ("Create once. Be visually recognizable anywhere.") and authenticate.

Key elements:

- Hero with before/after example: single CTA "Create My Identity"
- Clerk sign-in/sign-up component

User flow:

1. User clicks CTA
2. System shows sign-up
3. User completes sign-up and age confirmation

States:

- Empty state: n/a

- Loading state: skeleton while auth resolves
- Error state: inline plain-language auth error with retry

Consent

Purpose: Obtain explicit likeness consent.

Key elements:

- Short plain-language summary with expandable full policy
- Required attestation checkbox; separate optional training opt-in (off)
- "Continue" disabled until attestation is checked

User flow:

1. User reads the summary
2. User checks attestation
3. System records consent and opens the upload screen

States:

- Empty state: n/a
- Loading state: spinner on submit
- Error state: "We couldn't save your consent. Nothing was uploaded. Try again."

Photo Upload

Purpose: Collect 10–25 validated photos.

Key elements:

- Drag-and-drop zone and "Take photo" button on mobile
- Photo requirement guide (lighting, angles, no sunglasses, no other people)
- Grid of thumbnails with per-image status chips and fix hints
- Counter "12 of 10–25 valid photos"

User flow:

1. User adds photos
2. System validates each within 5 seconds
3. User replaces rejected photos
4. User clicks "Create identity" once the minimum is met

States:

- Empty state: guide plus examples of good/bad photos
- Loading state: per-file progress bars
- Error state: per-file reason with one-click replace

Identity Review

Purpose: Approve or retry the identity.

Key elements:

- Processing status with time estimate
- Four-image identity check preview
- Buttons: Approve, Retry with new photos, Request review

User flow:

1. System finishes processing
2. User compares previews to themselves
3. User approves; starter credits are shown

States:

- Empty state: n/a
- Loading state: progress with estimated ready time; user can leave and is notified by email
- Error state: "Identity couldn't be created. Retry is free."

World Explorer & Create

Purpose: Pick a world and generate.

Key elements:

- Category tabs, search, world cards with credit estimate
- Generation panel: prompt, aspect ratio, variations, identity strength, live credit cost
- "Generate" button showing the exact cost

User flow:

1. User selects a world
2. User adjusts options
3. User confirms cost and generates
4. System shows job progress, then outputs

States:

- Empty state: featured worlds for a first-time user
- Loading state: staged progress labels
- Error state: failure message, "Retry", and a line confirming credits were refunded

Library

Purpose: Find, organize and reuse assets.

Key elements: grid, filters, project sidebar, favorites, bulk select

User flow:

1. User opens library
2. User filters or opens an asset
3. User remixes, animates, exports or deletes

States:

- Empty state: "No creations yet" with a CTA to World Explorer
- Loading state: skeleton grid
- Error state: retry banner

Wallet & Plans

Purpose: Show balance and let users buy credits.

Key elements: balance breakdown by type, expiration dates, ledger table, plan cards, credit packs

States:

- Empty state: starter credit explanation
- Loading state: skeleton
- Error state: "Payment could not be completed" with Stripe's message

Settings & Privacy

Purpose: Control identity, consent and data.

Key elements: consent status and withdraw button, training opt-in toggle, delete identity, delete account, export data

States:

- Error state: deletion in progress banner with deadline

10. Data Models

User

Field	Type	Required	Description
id	UUID	Yes	Primary key
clerk_user_id	string	Yes	External auth ID
email	string	Yes	Account email
display_name	string	No	Optional
age_gate_confirmed_at	timestamp	No	Set when 18+ confirmed
account_status	enum	Yes	active, suspended, deleted
created_at	timestamp	Yes	Creation timestamp

Relationships:

- One-to-many to Workspace

Indexes:

- clerk_user_id (unique), email (unique)

Workspace

Field	Type	Required	Description
id	UUID	Yes	Primary key
owner_id	UUID	Yes	FK to User
name	string	Yes	Display name
plan_id	string	Yes	free, creator, pro
created_at	timestamp	Yes	Creation timestamp

Relationships:

- Belongs to User; has one CreditWallet; has many Projects and IdentityProfiles

Indexes:

- owner_id

ConsentRecord

Field	Type	Required	Description
id	UUID	Yes	Primary key
user_id	UUID	Yes	FK to User
identity_id	UUID	No	Set once an identity is created
policy_version	string	Yes	Version accepted
attestation_text	text	Yes	Exact text shown
allow_model_training	boolean	Yes	Default false
ip_hash	string	Yes	Hashed IP address
created_at	timestamp	Yes	Immutable
withdrawn_at	timestamp	No	Set on withdrawal

Relationships:

- Belongs to User; has one IdentityProfile

Indexes:

- user_id, identity_id

IdentityProfile

Field	Type	Required	Description
id	UUID	Yes	Primary key
workspace_id	UUID	Yes	FK to Workspace
consent_id	UUID	Yes	FK to ConsentRecord
name	string	Yes	Private label

Field	Type	Required	Description
status	enum	Yes	draft, processing, review_required, approved, failed, suspended, deletion_in_progress, deleted
identity_version	integer	Yes	Starts at 1
provider_refs	jsonb	No	Encrypted provider identity references
quality_score	float	No	Automated quality result
created_at	timestamp	Yes	Creation timestamp
approved_at	timestamp	No	Approval timestamp

Relationships:

- Belongs to Workspace; has many IdentitySources and GenerationJobs

Indexes:

- workspace_id, status

IdentitySource

Field	Type	Required	Description
id	UUID	Yes	Primary key
identity_id	UUID	No	FK to IdentityProfile
asset_id	UUID	Yes	FK to MediaAsset
validation_status	enum	Yes	pending, valid, rejected
rejection_reason	string	No	Machine code
quality_score	float	No	Per-image score

Relationships:

- Belongs to IdentityProfile and MediaAsset

Indexes:

- identity_id, validation_status

WorldTemplate

Field	Type	Required	Description
id	UUID	Yes	Primary key
category	string	Yes	e.g. cinematic
title	string	Yes	Display title
prompt_template	text	Yes	Versioned template
settings_schema	jsonb	Yes	Allowed variables and ranges
safety_policy	string	Yes	Rating and constraints
version	integer	Yes	Increments on change

Field	Type	Required	Description
credit_estimate	integer	Yes	Default cost
active	boolean	Yes	Published flag

Relationships:

- Referenced by GenerationJob

Indexes:

- category, active

Project

Field	Type	Required	Description
id	UUID	Yes	Primary key
workspace_id	UUID	Yes	FK to Workspace
identity_id	UUID	No	Default identity
name	string	Yes	Project name
visibility	enum	Yes	private, link, unlisted, public
created_at	timestamp	Yes	Creation timestamp

Relationships:

- Belongs to Workspace; has many GenerationJobs

Indexes:

- workspace_id

GenerationJob

Field	Type	Required	Description
id	UUID	Yes	Primary key
project_id	UUID	No	FK to Project
identity_id	UUID	Yes	FK to IdentityProfile
identity_version	integer	Yes	Version used
world_id	UUID	No	FK to WorldTemplate
world_version	integer	No	Template version used
type	enum	Yes	image, upscale, animation
status	enum	Yes	queued, preparing_identity, generating, enhancing, completed, failed, cancelled
prompt	text	No	User prompt
settings	jsonb	Yes	Controls used
provider	string	Yes	e.g. higgsfield

Field	Type	Required	Description
model_version	string	Yes	Provider model
credit_cost	integer	Yes	Credits reserved/charged
provider_cost_usd	numeric	No	Actual provider cost
retry_count	integer	Yes	Default 0
idempotency_key	string	Yes	Unique per user
error_code	string	No	Failure code
created_at	timestamp	Yes	Creation timestamp
completed_at	timestamp	No	Completion timestamp

Relationships:

- Belongs to IdentityProfile; has many MediaAssets

Indexes:

- (identity_id, created_at), status, idempotency_key (unique)

MediaAsset

Field	Type	Required	Description
id	UUID	Yes	Primary key
job_id	UUID	No	FK to GenerationJob
owner_id	UUID	Yes	FK to User
type	enum	Yes	source_photo, image, video
storage_key	string	Yes	S3 key
visibility	enum	Yes	private, link, unlisted, public
provenance	jsonb	No	C2PA metadata
favorite	boolean	Yes	Default false
created_at	timestamp	Yes	Creation timestamp
deleted_at	timestamp	No	Soft delete marker, purged within 24 h

Relationships:

- Belongs to GenerationJob and User

Indexes:

- (owner_id, created_at), job_id

CreditWallet

Field	Type	Required	Description
id	UUID	Yes	Primary key

Field	Type	Required	Description
workspace_id	UUID	Yes	FK to Workspace
available_credits	integer	Yes	Never below 0
reserved_credits	integer	Yes	Held for running jobs

Relationships:

- Has many CreditLedgerEntries

Indexes:

- workspace_id (unique)

CreditLedgerEntry

Field	Type	Required	Description
id	UUID	Yes	Primary key
wallet_id	UUID	Yes	FK to CreditWallet
type	enum	Yes	grant, purchase, reservation, settlement, refund, expiration, adjustment
credit_type	enum	Yes	purchased, bonus, subscription
amount	integer	Yes	Signed
reference_id	string	No	Job ID, Stripe event ID, etc.
expires_at	timestamp	No	Expiration
created_at	timestamp	Yes	Append-only

Relationships:

- Belongs to CreditWallet

Indexes:

- (wallet_id, created_at), reference_id (unique where type needs idempotency)

Referral

Field	Type	Required	Description
id	UUID	Yes	Primary key
referrer_id	UUID	Yes	FK to User
referred_user_id	UUID	Yes	FK to User
status	enum	Yes	pending, activated, rejected, held
reward_entry_id	UUID	No	FK to CreditLedgerEntry
risk_flags	jsonb	No	Fraud signals

Relationships:

- Links two Users

Indexes:

- referrer_id, referred_user_id (unique)

SafetyCase

Field	Type	Required	Description
id	UUID	Yes	Primary key
asset_id	UUID	No	FK to MediaAsset
reporter_id	UUID	No	FK to User
category	string	Yes	e.g. impersonation
status	enum	Yes	open, in_review, resolved, appealed
resolution	text	No	Outcome
created_at	timestamp	Yes	Creation timestamp

Relationships:

- Belongs to MediaAsset

Indexes:

- status, asset_id

AuditEvent

Field	Type	Required	Description
id	UUID	Yes	Primary key
actor_id	UUID	No	User or admin
action	string	Yes	e.g. identity.deleted
target_type	string	Yes	Entity type
target_id	string	Yes	Entity ID
metadata	jsonb	No	Context, never raw biometric data
created_at	timestamp	Yes	Append-only

Relationships:

- References any entity by type and ID

Indexes:

- (target_type, target_id), actor_id

11. Integration Points

Higgsfield

Purpose: Image and video generation, and identity-consistent workflows.

Integration type: REST API / SDK behind a provider adapter interface

Data exchanged:

- Inbound: generated asset URLs, job status, cost metadata
- Outbound: prompt, settings, identity reference or source images, aspect ratio, duration

Authentication: API key held server-side only

Rate limits: Provider-defined; adapter enforces a concurrency cap and backs off with jitter

Fallback behavior: Retry up to 2 times, then route to Krea if the workflow is supported, else fail and refund

Krea

Purpose: Secondary image/video generation and enhancement.

Integration type: REST API behind the same adapter interface

Data exchanged:

- Inbound: generated asset URLs, job status
- Outbound: prompt, settings, reference images

Authentication: API token, server-side only

Rate limits: Provider-defined; per-model concurrency cap in config

Fallback behavior: Mark provider unhealthy after 5 consecutive failures; route to primary; surface "high demand" status in UI

Stripe

Purpose: Subscriptions, credit packs, invoices.

Integration type: SDK, Checkout, Customer Portal, Webhooks

Data exchanged:

- Inbound: payment, subscription and invoice events
- Outbound: customer ID, price IDs, checkout session parameters

Authentication: Secret key server-side; webhook signing secret

Rate limits: Stripe default

Fallback behavior: Webhooks idempotent by event ID; nightly reconciliation job compares Stripe to ledger and alerts on drift

Clerk

Purpose: Authentication and session management.

Integration type: SDK and JWT verification

Data exchanged:

- Inbound: user identity, session tokens
- Outbound: none beyond configuration

Authentication: Publishable key in client, secret key server-side

Rate limits: Clerk default

Fallback behavior: If Clerk is unavailable, existing valid JWTs continue to work until expiry; new logins show a status message

Moderation provider

Purpose: Prompt and image safety checks.

Integration type: REST API

Data exchanged:

- Inbound: category scores and a block/allow decision
- Outbound: prompt text or output image

Authentication: API key server-side

Rate limits: Provider-defined

Fallback behavior: Fail closed: if moderation is unavailable, generation is paused and credits are not reserved

12. Edge Cases & Error Handling

Edge Cases

Scenario	Expected Behavior
User uploads photos containing another person	Image rejected with reason "More than one person detected"; counts as invalid
User tries to create an identity of a celebrity or minor	Blocked or sent to manual review; no credits or processing consumed; audit event logged
User double-clicks Generate	Idempotency key ensures one job and one reservation
Provider fails after charging us	User credits refunded automatically; provider cost logged for finance review
Worker crashes mid-job	Job is requeued by timeout; reservation persists until settle or refund
User deletes identity while a job is running	Running jobs cancelled, credits refunded, outputs discarded
Stripe webhook arrives twice or out of order	Processed once by event ID; ordering handled by event timestamps and state checks
Consent withdrawn after assets were shared	Share links revoked immediately; assets removed per deletion policy
Insufficient credits at confirm time	402 with a link to buy credits; no job created
Referred user uses same device as referrer	Referral held for review, no reward issued

Scenario	Expected Behavior
User uploads HEIC from iPhone	Converted server-side to JPEG for validation; original retained in encrypted storage
Moderation blocks a benign creative prompt	User sees reason and suggested rewrite, can appeal via "Request review"

Error Handling Strategy

- **User-facing errors:** Plain language, specific cause, one recovery action; never expose provider names or raw errors
- **System errors:** Structured JSON logs with request/job ID; alerts on error rate above 5% per provider over 10 minutes
- **Retry logic:** Provider calls retry twice with exponential backoff (2s, 8s); webhook processing retries via queue; user-initiated retries reuse unchanged inputs
- **Graceful degradation:** Provider outage disables only affected job types with a status banner; library, wallet, export and deletion keep working

13. Testing Requirements

Unit Tests

- Credit service: reserve, settle, refund, expire, negative-balance prevention, idempotency
- Prompt/template builder: variable substitution, versioning, safety constraints
- Photo validation rules: each rejection reason
- Referral eligibility and fraud rules
- Consent record immutability

Integration Tests

- Stripe webhook handler: duplicate, out-of-order, invalid signature
- Provider adapters against mock servers: success, timeout, 429, malformed response, fallback routing
- Authorization: every endpoint rejects cross-account access (403/404)
- Deletion pipeline: identity deletion removes storage objects, provider references and share links

E2E Tests

- Sign up → age gate → consent → upload 12 photos → create identity → approve → first generation → save → download
- Insufficient credits → purchase pack → generation succeeds
- Failed generation → credits refunded → retry succeeds
- Identity deletion → generation blocked → assets gone within SLA

Performance Tests

- 200 concurrent users creating image jobs: API p95 under 400 ms, queue wait p90 under 20 s
- 25-photo upload on a throttled 10 Mbps connection completes without timeout
- Library with 5,000 assets filters in under 2 seconds

14. Implementation Notes for AI

Build Order

1. PostgreSQL schema and migrations (User, Workspace, ConsentRecord, CreditWallet, CreditLedgerEntry, AuditEvent first)
2. Auth integration (Clerk), age gate, workspace bootstrap
3. Credit service with transactional reserve/settle/refund and full unit tests
4. Consent and upload endpoints, pre-signed S3 upload, photo validation worker
5. Identity creation job, provider adapter interface, Higgsfield adapter, admin review queue
6. Generation endpoints, queue workers, moderation integration, status polling/SSE
7. Frontend: landing, consent, upload, identity review, create screen, library, wallet
8. World templates catalog and World Explorer
9. Deletion pipeline and Safety Case/admin console
10. Stripe subscriptions and credit packs
11. Animation, share pages, provenance metadata, referrals
12. Krea adapter and routing/fallback

File Structure Suggestion

```
/apps
  /web                # Next.js app
    /app
    /components
    /lib
  /api                # Fastify API
    /routes
    /services         # credits, identity, generation, moderation, referrals
    /providers        # higgsfield.ts, krea.ts, adapter.ts
    /workers          # identity, generation, animation, refunds, deletion
    /db               # prisma schema and migrations
/packages
  /shared             # Zod schemas and types shared by web and api
  /config             # credit costs, plans, world catalog seeds
/tests
  /e2e
```

Critical Implementation Details

- **Credits are transactional:** Reserve credits and create the job in the same database transaction. Settle or refund in a worker with a row-level lock on the wallet. Never compute balance by summing in application code without the lock.
- **Idempotency everywhere money or jobs are created:** Require `idempotency_key` on generation and animation requests and enforce a unique index. Webhook handlers store processed Stripe event IDs.
- **Provider abstraction:** Define one `ImageProvider` and `VideoProvider` interface (`submit`, `poll`, `cancel`, `estimateCost`). Business logic never imports provider SDKs directly. Log provider, model version and cost on every job.
- **Biometric data handling:** Source photos and provider identity references are the most sensitive data in the system. Store in a dedicated bucket and table with encryption, separate access role, no analytics joins, and no logging of image content or embeddings. Use signed URLs with short expiry.
- **Deletion is a first-class workflow:** Build deletion as an auditable state machine (`requested` → `generation_disabled` → `storage_purged` → `provider_purged` → `complete`) with retries, and test it from day one.

- **Fail closed on safety:** If moderation or consent verification fails or is unavailable, do not generate and do not charge.
- **Config over code:** Credit costs, plan allowances, starter grants and expiry rules live in config/DB, not constants.
- **Dates:** Store UTC, display in the user's timezone.
- **Never expose** provider keys, internal identity references or other users' IDs in client payloads.

Code Style Preferences

- TypeScript strict mode, no any in service code
- Zod for request/response validation, shared between web and API
- kebab-case file names, PascalCase components, camelCase functions, snake_case DB columns
- One service per domain; routes stay thin

Libraries to Use

- Prisma for ORM — type-safe migrations
- BullMQ for queues — retries, delays and visibility
- TanStack Query for data fetching — caching and polling job status
- Zod for validation — single schema source
- Sharp for image checks — fast server-side processing
- Vitest and Playwright for tests

Libraries to Avoid

- Provider SDKs imported directly into route handlers — breaks the abstraction layer
- Client-side-only validation for credits or permissions — must be server-enforced
- Analytics SDKs that auto-capture page content on identity or upload screens — privacy risk

Common Pitfalls

- **Double charging:** Always route credit changes through the credit service; add a test that fires concurrent reserve calls
- **Orphaned data after deletion:** Track every storage key and provider reference in the database so deletion can enumerate them
- **Signed URL leaks:** Keep expiries short and never store signed URLs; generate on read
- **Misleading cost previews:** Compute cost server-side with the same function used at reservation time
- **Over-trusting provider output:** Always run output moderation and validate that the asset exists before marking a job completed

Testing Approach

- Write tests first for the credit service, authorization checks, consent enforcement and the deletion state machine
- Use mocked provider servers for all automated tests; run a small live smoke test per provider on a schedule
- Skip snapshot tests for UI layout; test behavior and accessibility instead
- Use Vitest for unit/integration and Playwright for E2E

15. Open Questions

1. Which jurisdictions are in scope at alpha, and has counsel cleared biometric-privacy compliance (e.g., Illinois BIPA, Texas, Washington, EU/UK GDPR) for each?
2. Do users own generated assets outright, or receive a license with plan-based commercial limits?

3. Are source photos retained after identity creation, or can users choose "train, then delete originals"?
 4. What is the objective fidelity threshold for "approved" (automated score and user rating)?
 5. Which provider workflows allow commercial use and support deletion requests for identity references?
 6. What is the maximum acceptable generation cost per active user and per video job?
 7. Which share setting should be the default beyond the private-link default assumed here?
 8. What proof-of-consent process is required before agency/client identities (post-MVP)?
-

16. Release Criteria (Closed Alpha)

- ☐ All P0 features (FR-001 to FR-007) meet their acceptance criteria
- ☐ Authorization, deletion and credit-ledger tests pass in CI
- ☐ Consent, privacy, acceptable-use, retention and refund policies are published and linked in product flows
- ☐ Moderation, reporting, suspension, takedown and identity deletion are operable from the admin console
- ☐ Generation success rate $\geq 95\%$ and p90 image latency under 90 seconds over a 7-day internal test
- ☐ Identity fidelity rating of 4+ from at least 65% of internal testers
- ☐ Legal review of biometric and privacy requirements is complete for the launch jurisdiction
- ☐ Analytics events implemented and validated for the full funnel